

Corso di Laurea Magistrale in Ingegneria Informatica - A. A. 2023/2024
Ingegneria del Software

Progetto O.C.M.
Open Clinic Manager



Studente:

Salvatore Alonzo (Matricola: 1000052471)

Prefazione

Il seguente documento presenta una descrizione relativa allo sviluppo dell'applicativo Open Clinic Manager implementata durante il corso di Ingegneria del Software. Il software è stato sviluppato in linguaggio Java sfruttando l'ambiente di sviluppo Eclipse, mentre la progettazione in UML è stata realizzata tramite il software Astah Professional. Verranno qui presentate solamente le versioni finali di ciascun elaborato ottenute al termine di tutte le fasi di progettazione.

Infine vengono descritte le fasi conclusive di testing e di refactoring del programma, con cui sostanzialmente sono state apportate delle migliorie al codice.

Indice

1 Ideazione e analisi dei requisiti	pag. 5
1.1 Introduzione	pag. 5
1.2 Requisiti	pag. 5
1.3 Obiettivi e casi d'uso	pag. 6
1.4 Modello dei casi d'uso	pag. 7
1.5 Specifiche Supplementari.	pag. 10
1.6 Regole di business.	pag. 11
1.7 Glossario	pag. 11
2 Analisi Orientata agli Oggetti	pag. 11
2.1 Introduzione	pag. 11
2.2 UC1 – Prenotazione visita.	pag. 12
2.2.1 Modello di dominio – UC1.	pag. 12
2.2.2 SSD e Contratti – UC1	pag. 13
2.3 UC2 – Gestione pagamenti	pag. 15
2.3.2 SSD e Contratti – UC2	pag. 16
3 Testing	pag. 17
3.1 Introduzione.	pag. 17
3.2 Individuazione dei casi di test e Testing Unitario	pag. 17

1 Ideazione e analisi dei requisiti

1.1 Introduzione

Lo scopo della fase di ideazione è quello di effettuare un'indagine al fine di ricavare un'idea generale del progetto da realizzare e quindi ottenere delle stime di fattibilità tenendo conto dei tempi e delle risorse necessarie.

Per capire al meglio come sviluppare e comprendere i requisiti del progetto, nella fase di ideazione sono stati presi in considerazione i seguenti documenti: Modello dei Casi d'Uso, Documento di Visione, Specifiche Supplementari, Regole di Business e Glossario.

1.2 Requisiti

Il titolare di una clinica richiede la realizzazione di un software che permetta la gestione delle visite cliniche. Il software deve rappresentare uno strumento integrato che segua il processo di prenotazione in tutte le sue fasi, a partire dalla gestione dei medici e pazienti.

In particolare l'operatore di sala deve poter:

- Registrare i dati dei medici e dei pazienti
- Gestire le prenotazioni delle visite
- Gestire le priorità dei medici
- Consultare i dati acquisiti
- Gestire i pagamenti

1.3 Obiettivi e casi d'uso

Analizzando i requisiti riportati nel paragrafo precedente, sono stati individuati l'attore principale a cui è destinato il sistema e gli obiettivi che egli intende portare a termine; da queste informazioni infine sono stati ricavati i casi d'uso principali.

Attore	Obiettivo	Caso d'Uso
Operatore	Visita medica	UC1: Prenotazione visita
Operatore	Pagamento della prestazione	UC2: Gestione pagamenti
Operatore	Inserimento e lista dei pazienti	UC3: Gestione pazienti (CRUD, Inserimento e lista)
Operatore	Inserimento e lista dei medici	UC4: Gestione medici (CRUD, Inserimento e lista)

I casi d'uso appena descritti si traducono graficamente nel seguente diagramma UML:

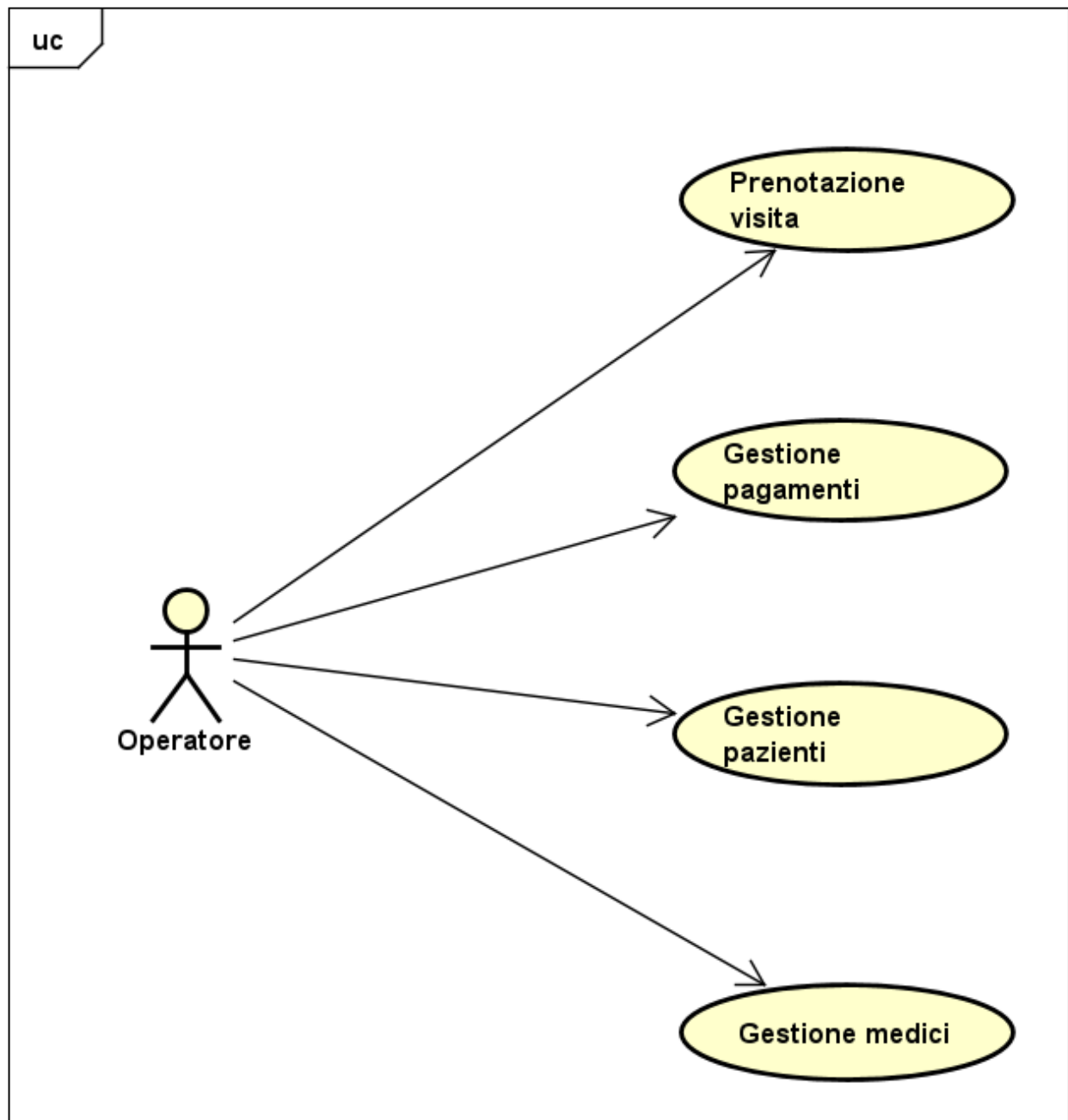


Figura 1 - Modello dei casi d'uso

1.4 Modello dei casi d'uso

Viene qui presentata una descrizione dei casi d'uso, in particolare, partendo dalla fase di ideazione in cui la maggior parte dei casi d'uso erano descritti in formato breve, seguendo il processo iterativo si è passati gradualmente ad una descrizione via via più dettagliata per la maggior parte dei casi d'uso, dunque si ha:

UC1: Prenotazione visita

Nome del caso d'uso	Prenotazione visita
Portata	Applicazione
Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	- Operatore: Riporta i dati acquisiti del paziente sull'applicativo il quale, basandosi sui dettagli riportati dal paziente, assegnerà una priorità alla visita ed il medico disponibile per competenza - Medico: Lo specialista deve avere la competenza necessaria ed avere lo slot libero per poter procedere alla visita
Precondizioni	Disponibilità del medico competente
Garanzia di successo	Il medico competente ha disponibilità per poter effettuare la visita
Scenario principale di successo	L'operatore seleziona il paziente dal software e lo assegna ad un medico, il software verifica che non ci siano problemi quali indisponibilità del medico e, nel caso non ci siano problemi, provvede alla prenotazione. Tale ciclo si ripeterà fin tanto che il paziente avrà delle visite da effettuare.
Estensioni	Nel caso in cui sopraggiunga un paziente con priorità maggiore in uno slot temporale con presente una precedente prenotazione, il paziente avente già prenotato perderà la priorità (caso di paziente in emergenza)
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Dipendente dal numero di visite richieste
Varie	

UC2: Gestione del pagamento

Nome del caso d'uso	Gestione del pagamento
Portata	Applicazione
Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	Operatore: Fornisce il totale dovuto per la prestazione al paziente

	Paziente: Paga la prestazione
Precondizioni	Devono essere presenti delle prenotazioni a carico del paziente
Garanzia di successo	Presenza di fondi da parte del paziente, compatibilità con i metodi di pagamento accettati dalla clinica
Scenario principale di successo	Deve esistere una prenotazione ed il paziente deve avere fondi a sufficienza per poter pagare la prestazione
Estensioni	Ci sono casi in cui si è esenti dal pagamento, se si hanno più di 65 anni
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Ogni qualvolta si richiede una visita
Varie	

UC3: Gestione pazienti (CRUD)

Nome del caso d'uso	Gestione pazienti (CRUD, Inserimento e lista)
Portata	Applicazione
Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	- Operatore: Acquisisce le generalità del paziente e le inserisci sull'applicativo - Paziente: Fornisce le generalità utili alla registrazione a sistema
Precondizioni	Il paziente necessita di una visita medica
Garanzia di successo	Il paziente fornisce le informazioni necessarie
Scenario principale di successo	Il paziente necessita di una visita specialistica per tanto recandosi in clinica fornisce i suoi dati. È possibile anche prendere visione dei dati depositati da parte dell'operatore
Estensioni	
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Ogni qual volta un nuovo paziente viene in clinica
Varie	

UC4: Gestione medici (CRUD)

Nome del caso d'uso	Gestione medici (CRUD, Inserimento e lista)
Portata	Applicazione

Livello	Obiettivo utente
Attore primario	Operatore
Parti interessate e Interessi	- Operatore: Registra a sistema il nuovo medico - Medico: Fornisce i dati
Precondizioni	Popolare le informazioni dei medici
Garanzia di successo	Il medico fornisce le informazioni necessarie
Scenario principale di successo	La clinica necessita dei medici specialisti per poter procedere alle visite, per tanto i medici vengono registrati a sistema.
Estensioni	
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Ogni qualvolta che uno specialista viene assunto dalla clinica.
Varie	

1.5 Specifiche Supplementari

Usabilità

- L'interfaccia grafica deve essere intuitiva e semplice anche per un utente non esperto.
- L'interazione con il sistema non deve presentare un elevato grado di complessità.

Affidabilità

- Il software sviluppato deve essere affidabile e deve poter mantenere i propri dati anche in caso di guasti (guasti elettrici, usura dell'hardware, attacchi informatici).

Vincoli hardware e software

- Per eseguire il software non ci sono particolari requisiti per il sistema operativo (è preferibile utilizzare Windows) purché sia presente la Java Virtual Machine a 64 bit con versione minima 1.8.

Vincoli di sviluppo del software

- Il software è scritto interamente in linguaggio Java

Aspetti legali

- Le tecnologie utilizzate per la progettazione e realizzazione del sistema proposto sono di tipo open source.
- O.C.M. viene rilasciato con licenza open source GPL v3.

1.6 Regole di business

Il software applica, nella gestione dei pagamenti, un algoritmo che si basa sulle regole qui sotto riportate:

- 1) Le visite verranno sottoposte ad un sovrapprezzo basate sull'esperienza del medico che effettuerà la visita (partendo da un'esperienza media che aggiunge al *prezzo base* di ogni visita 20€ mentre se il medico ha un'alta esperienza viene aggiunto un sovrapprezzo di 50€ per visita)
- 2) Si applica l'esenzione dal pagamento in base all'età (per i pazienti con un'età maggiore o uguale a 65 anni viene applicata l'esenzione del pagamento su qualunque visita)

1.7 Glossario

Vengono qui riportati i termini più significativi e le relative definizioni.

- Aggiungi: Il termine si riferisce al processo di aggiunta da parte dell'operatore di un elemento appartenente ad una delle tre categorie descritte sopra.
- Lista: Il termine si riferisce al processo di stampa a video da parte dell'operatore di tutti gli elementi appartenenti ad una delle tre categorie descritte sopra
- Vai al pagamento: Il termine si riferisce al processo di pagamento della visita

2 Analisi Orientata agli oggetti

2.1 Introduzione

Analizziamo adesso i casi d'uso mostrandone modello di dominio, diagramma di sequenza di sistema e contratti per ciascuno.

2.2 UC1 – Prenotazione visita

2.2.1 Modello di dominio – UC1

La rappresentazione del modello di Dominio è la seguente:

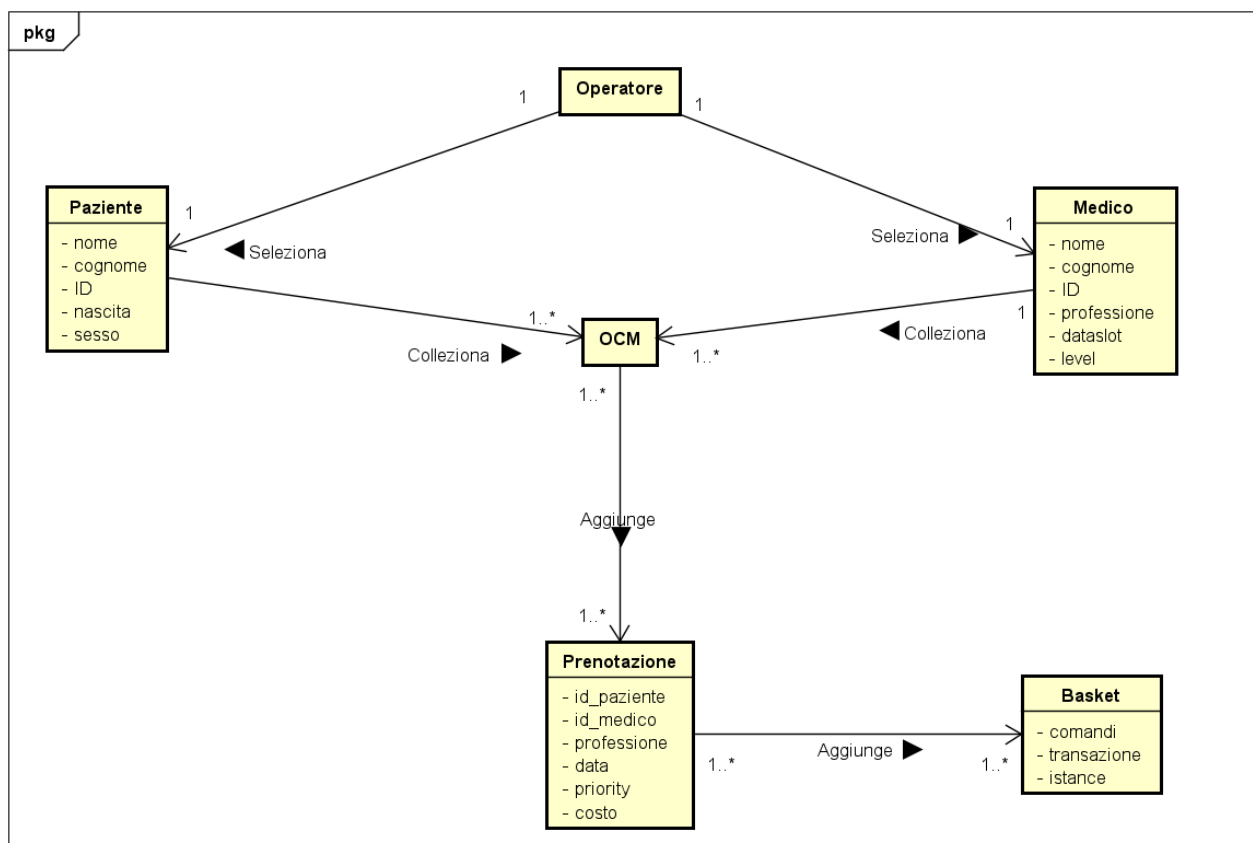


Figura 2 - Modello di dominio UC1

L'operatore tramite l'applicativo seleziona un medico e un paziente tramite una lista aggiunti in precedenza e, una volta fatto ciò, viene creato un nuovo oggetto prenotazione.

Una prenotazione viene composta dai seguenti elementi:

- 1) ID Paziente: contiene l'identificativo del paziente univoco
- 2) ID Medico: contiene l'identificativo del medico univoco
- 3) Professione: Indica la professione del medico
- 4) Data Prenotazione: Indica lo slot temporale per la visita
- 5) Priorità: Indica la priorità attribuita dall'operatore al paziente
- 6) Costo: Indica il costo della visita

2.2.2 SSD e Contratti – UC1

Procedendo con l'analisi Orienta agli Oggetti, il passo successivo è la creazione del Diagramma di Sequenza di Sistema (SSD) al fine di illustrare il corso degli eventi di input e di output per i vari casi d'uso esaminati in ciascuna iterazione. Inoltre le principali operazioni di sistema individuate negli SSD verranno descritte attraverso i Contratti.

Lo schema riportato qui sotto rappresenta la gestione delle prenotazioni:

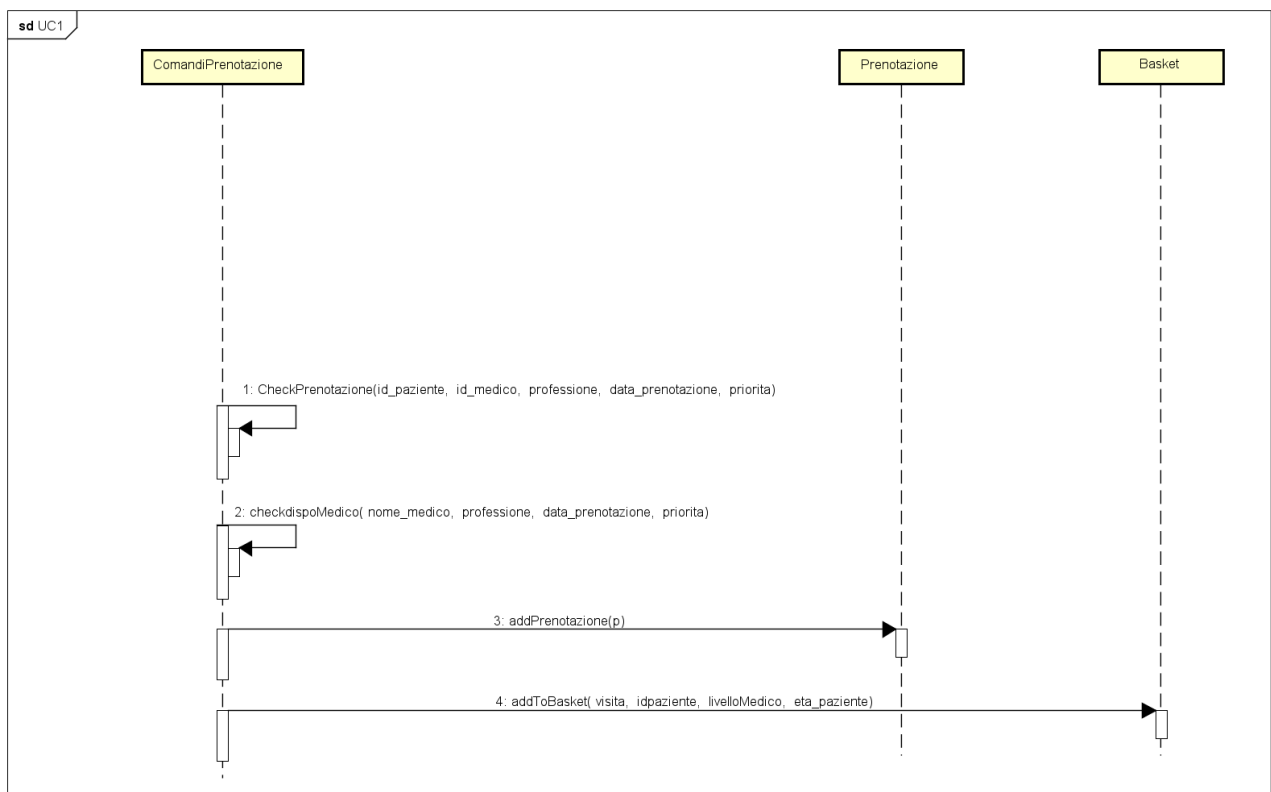


Figura 3 - SSD UC1

Contratto CO1: *CheckPrenotazione(id_paziente:String, id_medico:String, professione:String, data_prenotazione:String, priorita:String)*

Quest' operazione di sistema verifica che non vi siano prenotazioni presenti nello slot scelto

Operazione	CheckPrenotazione id_paziente:String, id_medico:String, professione:String, data_prenotazione:String, priorita:String)
Riferimenti	Caso d'uso: Prenotazione visita
Pre-condizioni	Stiamo aggiungendo una prenotazione
Post-condizioni	Verifico la disponibilità del medico

Contratto CO2: *checkdispoMedico(id_medico:String, professione:String, data_prenotazione:String, priorita:String)*

Quest' operazione di sistema verifica (dipendente dal precedente contratto) che non vi siano sovrapposizioni di pazienti negli slot prenotati in precedenza, a meno di emergenze (quindi se abbiamo emergenze)

Operazione	checkdispoMedico(id_medico:String, professione:String, data_prenotazione:String, priorita:String)
Riferimenti	Caso d'uso: Prenotazione visita
Pre-condizioni	Stiamo verificando che non vi siano prenotazioni per lo slot temporale selezionato
Post-condizioni	Aggiungo la prenotazione

Contratto C03: *addprenotazione(p:PrenotazioneBuilder)*

Qui inseriamo, tramite il builder delle prenotazioni, la prenotazione in una lista fornendo le varie informazioni quali id del medico, la sua professione, la data di prenotazione e la priorità attribuita.

Operazione	addprenotazione(p:PrenotazioneBuilder)
Riferimenti	Caso d'uso: Prenotazione visita
Pre-condizioni	Non ci sono prenotazioni nello slot temporale scelto o vi è una priorità maggiore.
Post-condizioni	Aggiungo la prenotazione

Contratto C04: addToBasket(visita:String, idpaziente:String, livelloMedico:String, eta_paziente:String)

In questa operazione inseriamo informazioni necessarie delle visite precedentemente prenotata nel carrello.

Operazione	addToBasket(visita:String, idpaziente:String, livelloMedico:String, eta_paziente:String)
Riferimenti	Caso d'uso: Prenotazione visita
Pre-condizioni	Abbiamo aggiunto la prenotazione
Post-condizioni	La prenotazione è stata aggiunta al carrello

2.3 UC2 – Gestione pagamenti

Tramite il sistema si seleziona il paziente e si va al menu di pagamento il quale restituirà le prenotazioni effettuate dal paziente col totale dovuto

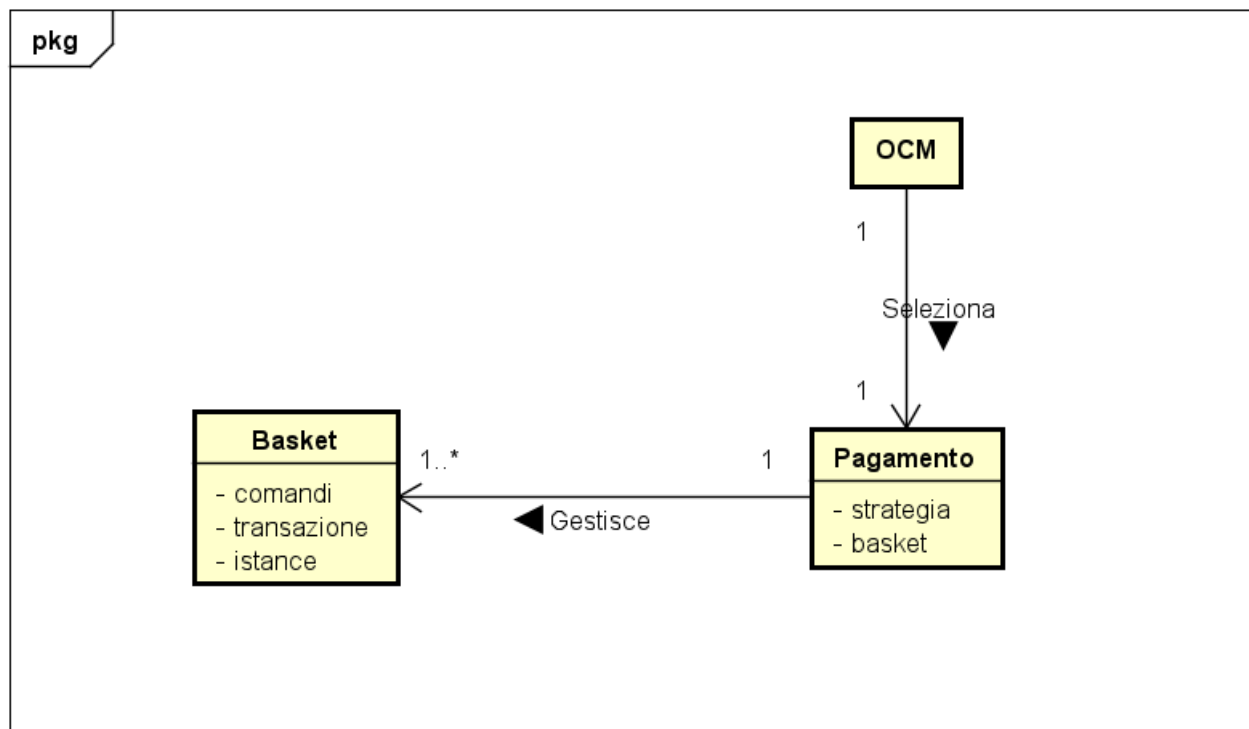


Figura 4 - Modello di dominio UC2

2.3.2 SSD e Contratti – UC2

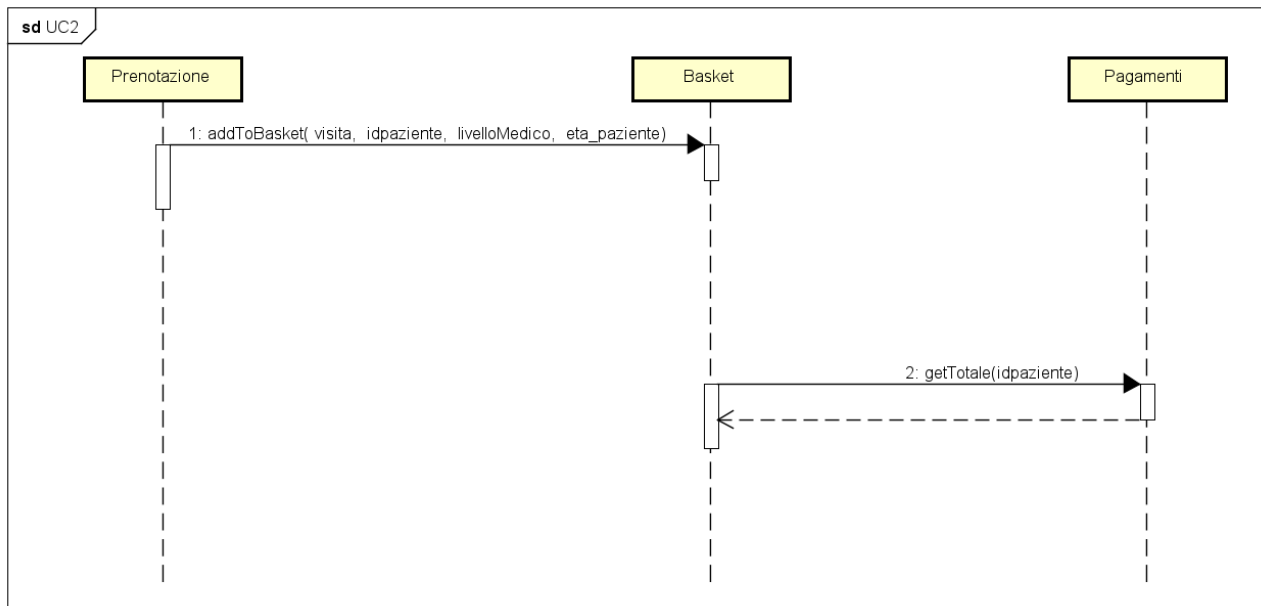


Figura 5 - SSD UC2

Contratto CO1: *addToBasket(visita:String, id_paziente:String, livello_medico:String, eta_paziente:String)*

Quest' operazione inizio il processo di aggiunta della prenotazione nel carrello.
Nello specifico, innesta i controlli per verificare se, rispettando le regole di business, vi siano pazienti con età che sia maggiore o uguale a 65 anni per poter applicare l'esonazione, verifica inoltre il livello di esperienza dei medici per poter applicare il sovrapprezzo al prezzo della visita bassa.

Operazione	addToBasket(visita:String, id_paziente:String, livello_medico:String, eta_paziente:String)
Riferimenti	Caso d'uso: Gestione pagamenti
Pre-condizioni	Prenotazioni effettuate
Post-condizioni	Aggiunte le prenotazioni nel carrello

Contratto CO2: *getTotale(idpaziente:String)*

In questa sezione otteniamo il totale dovuto dal paziente.

Operazione	getTotale(idpaziente:String)
Riferimenti	Caso d'uso: Gestione pagamenti
Pre-condizioni	Prenotazione nel carrello
Post-condizioni	Totale dovuto

3 Testing

3.1 Introduzione

Il testing è una fase essenziale del processo di sviluppo di un programma robusto ed efficiente, ne permette inoltre la riduzione in maniera sostanziale dei costi di manutenzione. La probabilità di rilevare malfunzionamenti è proporzionale al numero di test eseguiti, tuttavia è difficile testare un programma completamente, infatti le possibili combinazioni di valori di input da prendere in considerazione sono molteplici e non sempre possono essere riprodotti in un tempo ragionevole. Tuttavia un testing progettato accuratamente può rivelare comportamenti anomali ed indesiderati al fine di rendere il software realizzato funzionante e funzionale secondo le specifiche del committente.

I test si dividono principalmente in due famiglie:

- I test unitari (Unit Test) → Sono semplici test/prove che vanno a verificare la correttezza direttamente del codice, in ogni sua piccola parte. L'idea dello Unit Test in Java è quella di valutare ogni singolo metodo in funzione dei valori attesi. Esistono diversi tool per Unit Testing per i diversi linguaggi di programmazione, in particolare JUnit è il framework più diffuso attualmente per l'automazione del testing di unità di programmi Java.
- I test funzionali → Sono dei test che vanno a verificare che il sistema software nella sua completezza funzioni correttamente. Questi test trattano il sistema come se fosse una scatola nera alla quale danno degli input e verificano la correttezza degli output. Per eseguire il testing dell'applicazione realizzata, si è scelto di concentrarsi principalmente su test unitari eseguiti tramite l'ausilio di JUnit.

3.2 Individuazione dei casi di test e Testing Unitario

In una fase preliminare al testing, si è presa coscienza ed ispezione del codice

sorgente scritto al fine di operare una scelta sulle classi e sui metodi da testare. Il criterio scelto per individuare tali metodi su cui concentrare i test di unità è stato quello di dare priorità a tutti i metodi che si occupano dell'inserimento di nuove istanze le quali possono facilmente portare ad errori quantità e la creazione dei vari oggetti.

A seguito dell'esecuzione di questi test unitari con JUnit, si è passati a correggere le relative porzioni di codice che portavano al fallimento di alcuni test.

4.3 Test di Sistema

Si è scelto di eseguire dei test di sistema manuali lanciando l'applicazione e provando a portare a termine i principali casi d'uso con i rispettivi scenari alternativi.

A seguito di tali test non sono emersi grandi errori nel funzionamento e le prestazioni del sistema si sono mantenute piuttosto elevate, tuttavia sono state rilevate delle imperfezioni e anomalie nell'esecuzione di alcune funzionalità:

Gli aspetti principali testati sono stati:

- 1) Inserimento di un paziente
- 2) Inserimento di un medico
- 3) Inserimento di una prenotazione
- 4) Inserimento della prenotazione nel carrello
- 5) Check pagamento con e senza esenzione